Of course, Counting, Radix and Bucket sort take O(n) time, but ask for input restrictions and finally cannot sort all the input given.

In this algorithm, we use a hash table implicitly to operate onto input elements, and only by doing two scans we are able to sort any kind of input (no input restriction) provided to us.

Implementation: How does it work?

Now, let us find a better solution to this problem. Since our objective is to sort any kind of input, what if we get the maximum elements and create the hash? So, create a maximum hash and initialise it with all zeros. For each of the input elements, go to the corresponding position and increment its count. Since we are using arrays, it takes constant time to reach any location.

Note: All the code described in this algorithm was checked in 32-bit compilers (CodeBlocks) and in gcc-compilers.

Here is the main code implementation:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

void main()
{
long int arr[]={62,4,8,423,43,4,432,44,23,2,55,12,3};
int n=sizeof(arr)/sizeof(arr[0]) ;
int i, j, k,l;
int min=arr[0];
int max=arr[0] ;
for(i=1;i<n;i++)       // run for n times
  {
    if(arr[i]<min)
     min=arr[i] ;       // required for optimization
    else if(arr[i]>max)
     max=arr[i] ;
  }
int *hash=(int*)calloc((max+1), sizeof(int)); //should be max+1

for(i=0;i<n;i++ )
  {
   hash[arr[i]]++; // storing the count of occurances of element
  }
  printf("\n");

  for(i=0;i<max+1;i++)   // loop to read the hash table...
 {
  if(hash[i]>0)
  {
  printf("%d\t", i)  ; //print 1st time if the element has occured
  }
  j=hash[i];      /* this requires if the count of the element
            is greater than one(if there is duplicates
           element in the array) otherwise dupplicated element
           will not be displayed more than once.*/
  if(j==1 ||j==0)
  continue;
    while(j!=1)
    {          // here we are printing till count not equal to 1
    printf("%d\t", i) ;
    j--;
    }
 }
getch();
}
```

The time complexity of the above algorithm is O(n) and with only two scans, we are able to arrive at the result. Its space complexity is O(n), as it requires extra auxiliary space (almost the maximum elements in an array) to get the result.

How is it better than other existing sorting algorithms?

Let's check how this new algorithm compares with other existing sorting algorithms in terms of time complexity and speedup, flexibility and elegance of code.

I have already given you a glimpse of the Bubble sort and its optimisation. Now let's take a look at the time complexity of existing algorithms:

Selection sort	Best case- O(n)	Worst case-O(n2)
Bubble sort	Best case O(n)	Worst case-O(n2)
Insertion sort	Best case- O(n2)	Worst case-O(n2)
Merge sort	Best case- O(nlogn)	Worst case-O(nlogn)
Quick sort	Best case- O(nlogn)	Worst case-O(n2)
Radix sort	Best case- O(n)	Worst case-O(n)
Bucket sort	Best case- O(n)	Worst case-O(n)
Counting sort	Best case- O(n)	Worst case-O(n)

Note: Radix, Bucket and Counting sort make assumptions about the input provided, i.e., they cannot sort all the input provided. There is an input restriction, though (about which there will be no discussion in this article).

How the new sorting algorithm is better in terms of time-complexity and overhead

I am using *rand()*. The function *rand()* generates a pseudo-random integer number. This number will be in the range of 0 to RAND_MAX. The constant RAND_MAX is defined in the standard library *(stdlib)*.

```
#include<stdio.h>
#include<stdlib.h>
```